T14

Grepai: Semantic Search

Technical Intermediate

Finding code by intent, not by regex

Principle: search by intent

Native Grep (ripgrep) is ideal when you know the exact text to find. Grepai takes over when you are looking for “the code that handles X” without knowing the exact function or variable names. It relies on local embeddings via Ollama and the `nomic-embed-text` model, which keeps that indexing step local.

Search results returned to Claude can still enter the configured model provider’s context.

Measure value: compare retrieval accuracy, returned context size and accepted-task cost on representative searches.

The 5 main tools

`grepai_search`: search by natural language description. Short queries (1-3 keywords) yield better results than long phrases.

```
grepai_search("payment flow")
grepai_search("JWT validation")
grepai_search("rate limiting middleware")
```

`grepai_trace_callers`: finds all functions that call a given symbol.

`grepai_trace_callees`: lists what a function calls internally.

`grepai_trace_graph`: generates the full dependency graph around a symbol.

`grepai_index_status`: checks that the index is up to date before running searches.

Call graph in practice

The most powerful use case for grepai is dependency analysis. Before refactoring a function, knowing who calls it prevents silent regressions.

```
# Who calls validateJWT?
grepai_trace_callers("validateJWT")
→ ApiGateway, AdminPanel, UserController

# What does validateJWT do internally?
grepai_trace_callees("validateJWT")
→ decodeToken, checkExpiry, verifySignature
```

Recommended depth: 2 by default, 3 at most for complex analyses.

Grepai vs native Grep

Situation	Recommended tool
Exact pattern known (`"authenticate"`)	Native Grep
Search by concept or intent	Grepai
Call graph analysis	Grepai (`trace_callers`)
Multi-file refactoring	Grepai + Serena
Advanced regex on AST structure	ast-grep

Simple rule: if you can write the regex, use Grep. Otherwise, grepai.

Setup

```
# Install grepai
curl -sSL https://raw.githubusercontent.com/yoanbernabeu/grepai/main/install.sh | sh

# Initialize in the project (choose: ollama, nomic-embed-text)
grepai init

# Check status
grepai status
```

Prerequisites: Ollama installed with the `nomic-embed-text` model available. If “connection refused” error: `brew services start ollama`.

Best practices

Always call `grepai_index_status` at the start of a session if the codebase has changed since the last indexing. Limit results to 5 by default and increase only if the first results are not relevant. Grepai discovers files, then native Grep refines with a precise regex: both tools complement each other naturally.